

Jetbot self driving project

Group:

- Marcin Perek, nr. 160298
- Filip Podalak, nr. 160311
- Adam Mierzwicki, nr. 160318
- Dariia Shevchuk, nr. 161200
- Maria Pisaryk, nr. 161190

Our general approach

We recorded our dataset from a camera image on jetbot while driving manually, then a deep learning model is trained to predict jetbot controls and finally we use our trained model to infer controls from a robot camera and drive autonomously.

Dataset

Our initial attempts to record a dataset at the very beginning of the jetbot project classes went quite wrong because due to technical issues we were not able to record the videos properly. So our solution by then was to use the ready made dataset from the put jug. We tried training the model, but every approach seemed to result in poor generalization on the track.

At this point due to additional classes we decided to record the data once again and this time everything worked correctly. So as for the dataset our approach was to record videos with start and stop functionality to record only the best highlights and then aggregate the data. We recorded forward and turn controls, as well as left and right wheel speeds as backup, but in the end only forward and turn was used. We were able to get about 7500 frames of videos which was just enough to train the model to work correctly. To ensure that the model will generalize properly on a real track, we drove the jetbot bidirectionally and also included more exact curves to increase the abilities of the robot to turn, which actually enhanced the performance significantly.

Model training

Our initial approach was to build just a simple network starting with few convolutional layers and narrowing down to linear layers, having 2 outputs, the controls. Even though the approach was really simple and training of the model was taking just a few minutes, the performance of the model on driving the jetbot was already quite decent.

We wanted to have something more advanced. So we have chosen to use the ResNet-18 backbone and with linear layers head. By using it the performance enhanced significantly. As for outputs as we mentioned before there are forward and turn steering, so there are no analytical calculation steps after, like many other groups did. Our jetbot is deciding where to go 100% on its own.

One important feature we came up with is that the data is shifted forward, which means that the controls which we will predict are actually from the +5 frame. This allows to lower the inference lag on the robot and also made our model more robust in sharp turns.

At the very end, we thought of using a grayscale input. We modified our ResNet-18 model to include a convolutional input layer, so that it accepts a 1-channel input instead of the traditional 3-channel one ResNet expects. This ended up improving our results tremendously.

We trained our model for 30 epochs to ensure that it is not overfitting that much (unfortunately it did but in an acceptable way) and saved the model to torch format for inference. Unfortunately due to huge technical problems with installing onnxruntime we stick to the torch version.

Inference

For inference we just read the data from the camera and run the model to predict the controls. Initially we were experiencing some inference lags, especially visible on sharp turns. It was due to running inference inside the callback which was the major bottleneck and slowed the process, it turned out that using just a while loop was significantly faster and allowed our robot to drive smoothly.

Reproducibility

All changed code is inside github repository

<https://github.com/adammierzwicki/jetbot-project-robotics/>

Our changes were made to the notebooks directory, having our main model scripts and notebooks inside driver_model directory. All code and model weights are included here so self driving is ready to be run just after clone the repo. We also tested whether our approach is reproducible just by taking a different robot and running a pipeline again. It turned out that everything went well without any issues and the robot was driving on its own.

Problems encountered

The major problem we encountered was actually working with hardware. We spent almost 3 / 4 classes, plus additionally staying after classes several times to actually install dependencies properly and not mess up the environment, which happened multiple times. We found the behaviour and setup really inconsistent on different robots which made it much harder. We believe that without those issues we would have lots of time for new ideas on how to enhance our approach.

The model was oversensitive to camera angle. We found out that a correct angle of the camera was crucial to ensure proper jetbot driving. Even some small changes sometimes made our robot struggle with sharp turns.

On top of that, vibrant background elements off the track occasionally distracted the model, as processing all RGB channels created noise that drew attention away from the road.

Error analysis

While our approach started to take shape, it showed a few specific limitations. After 30 training epochs, the model showed signs of overfitting, likely memorizing visual features of the exact track rather than generalizing the visual of a road. It was also highly sensitive to the physical camera setup, slight changes in tilt caused struggles on turns, pointing to reliance on exact pixel positioning over robust spatial awareness. Abandoning the ONNX

runtime due to dependency conflicts forced us to use the heavier PyTorch model. Although our while-loop implementation and +5 frame shift significantly mitigated the lag, the model occasionally lacked the split-second responsiveness an optimized ONNX version would likely be able to provide.

Activation analysis

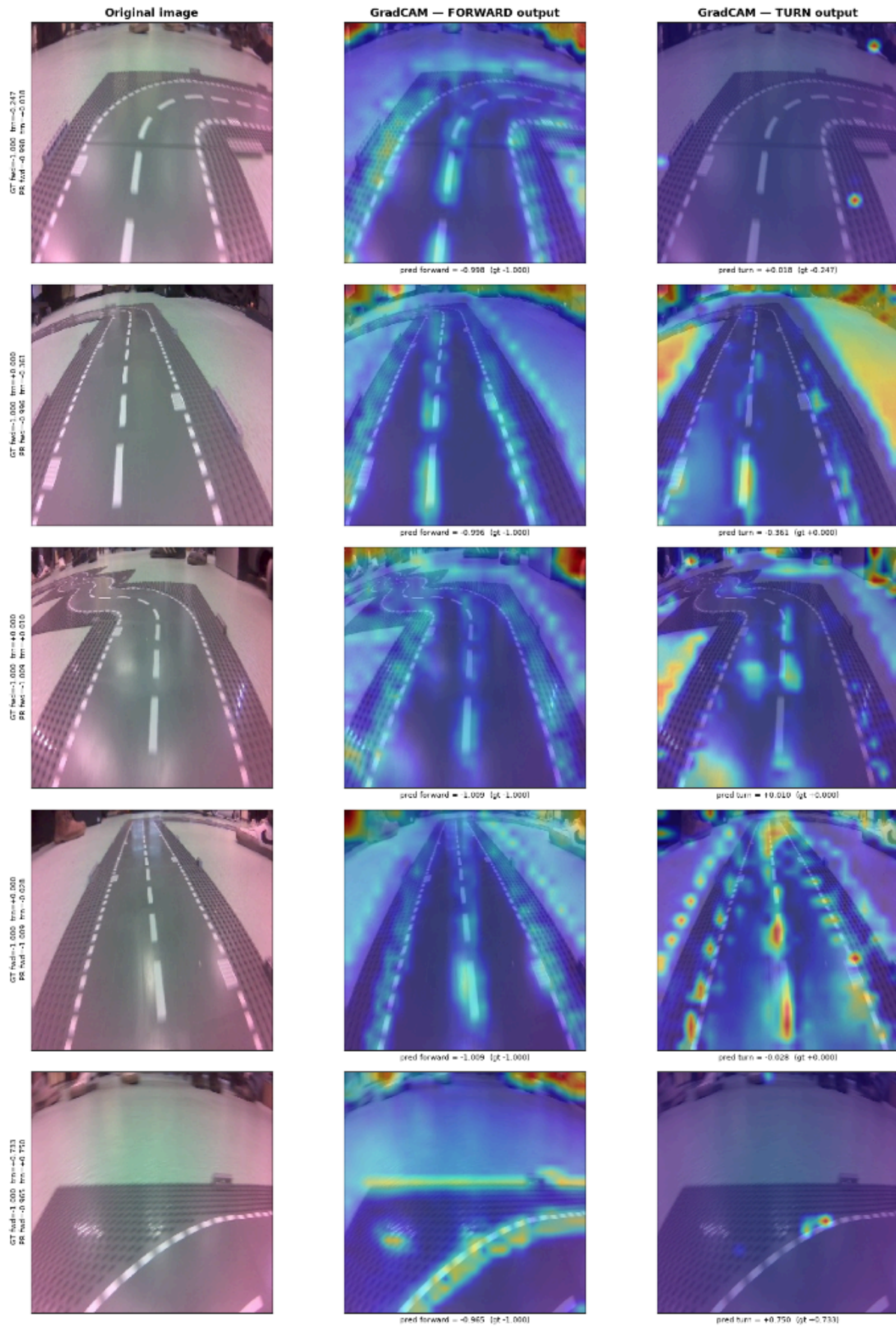
At the end we performed activation analysis using GradCam method to check which pixels in the image contribute the most to the output of our model. We tested the outputs of the 3rd convolutional layer in our grayscale model (last conv layer). On the images below we can see the result of the analysis where the first column is the original picture, second is the activation for forward signal and the last one is the activation of turn signal overlaid on top of the image. We can see that for the most part the network focuses its attention around lines on the track but sometimes most attention is drawn to background objects like wheels of the chair. In the turn signal we can also see that the model learned to look at the ground around the track which also makes sense to turn when the road is ending. Overall the activation pattern seems to be surprisingly correct for such a small model.

Conclusions

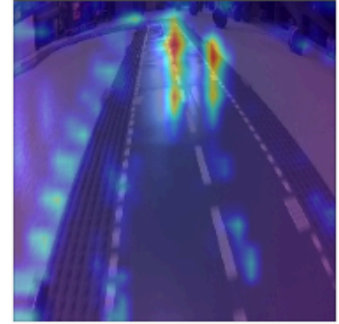
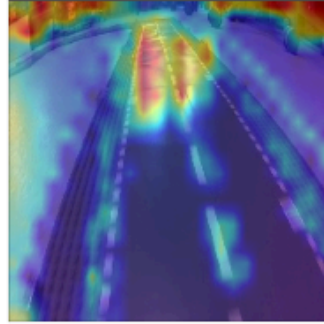
Overall, the project provided great hands-on experience with autonomous driving pipelines on edge hardware. By replacing standard analytical steering with a prepared model, the Jetbot decided its trajectory entirely on its own. One of our biggest takeaways is that data quality outweighs quantity. Recording our own dataset with added focus on turns and bidirectional driving proved highly effective. One of the last changes, our pivot to grayscale image processing, successfully minimized background distractions and streamlined data processing, proving that constraints breed creative engineering solutions.

We also learned that hardware integration and software dependencies tend to be the biggest bottlenecks in robotics, taking up far more time than mathematical adjustments. However, creative engineering solved significant real-world issues: shifting target data by +5 frames and using a simple while-loop eliminated critical inference lag. Despite technical difficulties, we achieved our goal of building a robust, reproducible pipeline. Successfully deploying our repository onto a completely different robot with no loss in performance proved this reliability, allowing us to consider future improvements like further inference optimization.

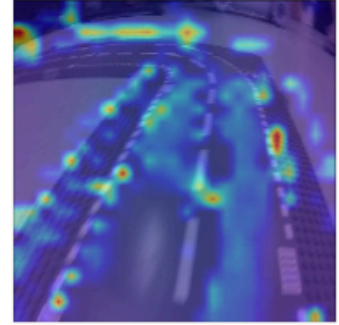
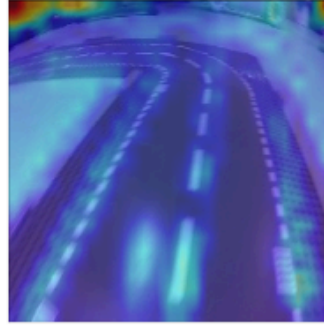
GradCAM Activation Analysis — driver_model_gray.pth



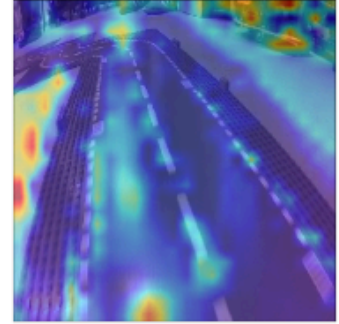
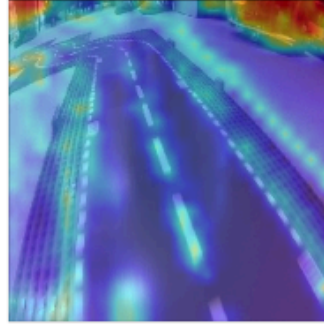
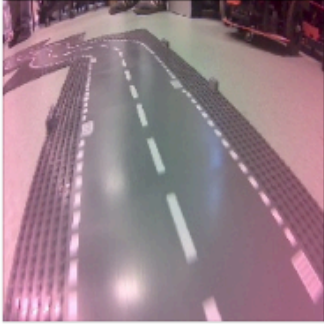
GT fwd=-1.000 turn=-2.000
PR fwd=-0.981 turn=-0.131



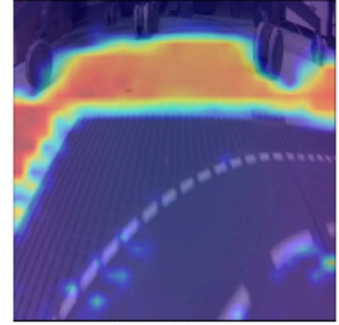
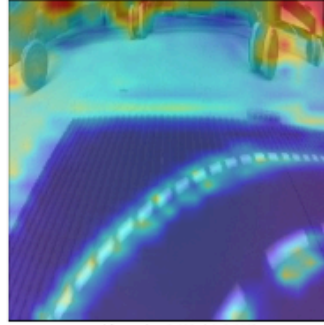
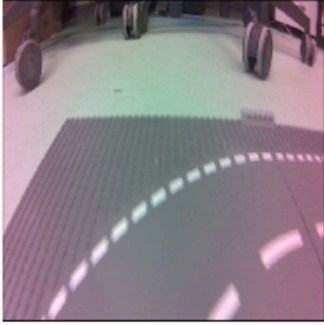
GT fwd=-1.000 turn=-2.561
PR fwd=-1.017 turn=-2.007



GT fwd=-1.000 turn=-0.000
PR fwd=-1.015 turn=-0.000



GT fwd=-1.000 turn=-0.000
PR fwd=-1.011 turn=-0.141



GT fwd=-1.000 turn=-0.000
PR fwd=-0.998 turn=-0.000

